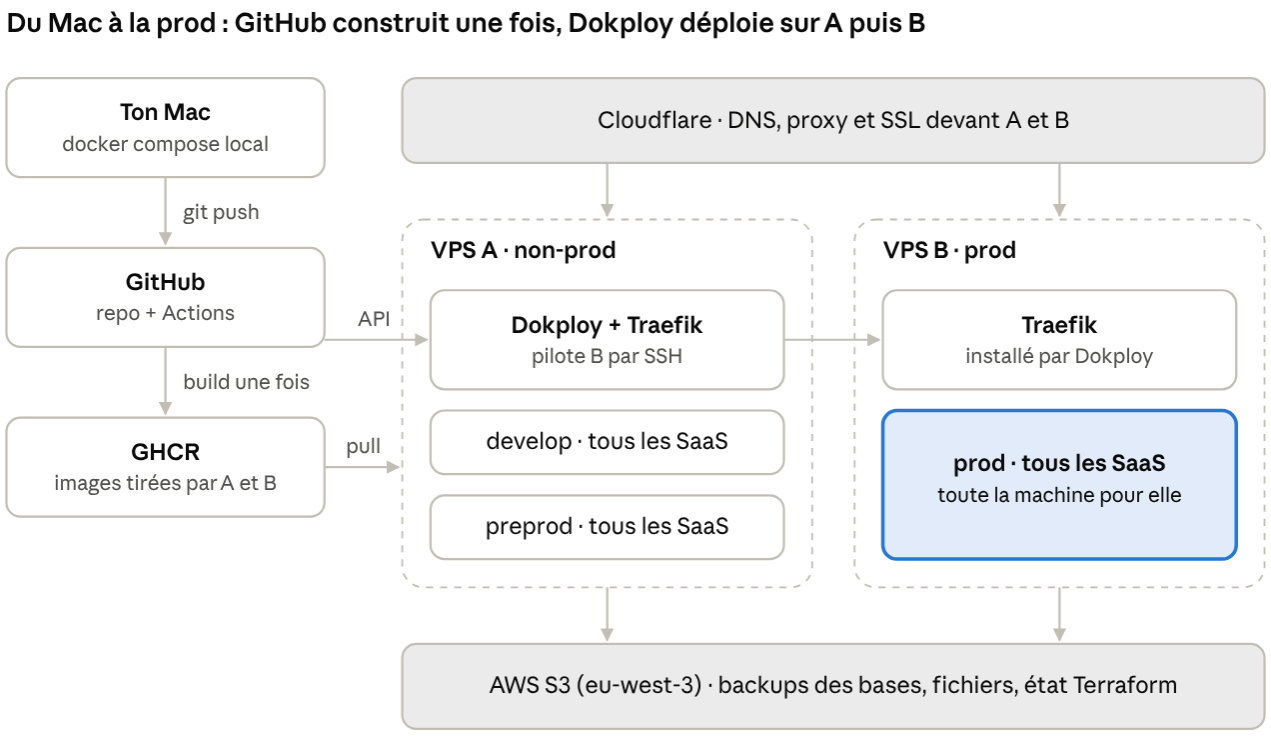
Plateforme SaaS — architecture & runbook

Oct 29, 2026 · @Kevin Kouomeu

Vue d'ensemble

Une fois la plateforme montée (une seule fois), lancer une idée de SaaS revient à créer un monorepo depuis le template et à lancer une commande : develop, preprod et prod sont en ligne avant même la première feature.



architecture globale · 2 VPS, GitHub, GHCR, Cloudflare, S3

Seule la prod vit sur B ; Dokploy, develop et preprod vivent sur A, et les deux serveurs sauvegardent vers S3.

Conventions de ce document. VPS A = serveur non-prod (Dokploy + develop + preprod). VPS B = serveur prod. `tonlab.fr` = ton domaine « plateforme » pour les outils (exemple). `supersaas.com` = un SaaS (exemple). `toi` = ton compte GitHub. Remplace ces noms par les tiens.

Les 6 principes qui tiennent tout le reste

1. **Plateforme ≠ produits.** Le socle (serveurs, DNS, registry, workflows, template, Terraform) se construit une fois ; chaque SaaS ne fait que le consommer.

2. **Deux serveurs, deux rôles.** Le VPS A héberge Dokploy et les environnements develop et preprod de tous tes SaaS. Le VPS B n'héberge que les prods, piloté à distance par Dokploy.
3. **Build une fois, promouvoir ensuite.** GitHub Actions construit chaque image une seule fois (tag = sha du commit). Preprod et prod reçoivent exactement l'image testée en develop.
4. **Les données ne bougent pas avec l'app.** Les bases sont des services séparés, jamais redéployés avec le code. Fichiers utilisateurs et backups vivent sur AWS S3, hors des serveurs.
5. **Convention plutôt que configuration.** Un dossier avec un Dockerfile = une image = un service, avec le même nom partout (repo, registry, compose, Dokploy).
6. **Tout ce qui se fait deux fois devient du code.** Terraform, workflows réutilisables, script de bootstrap.

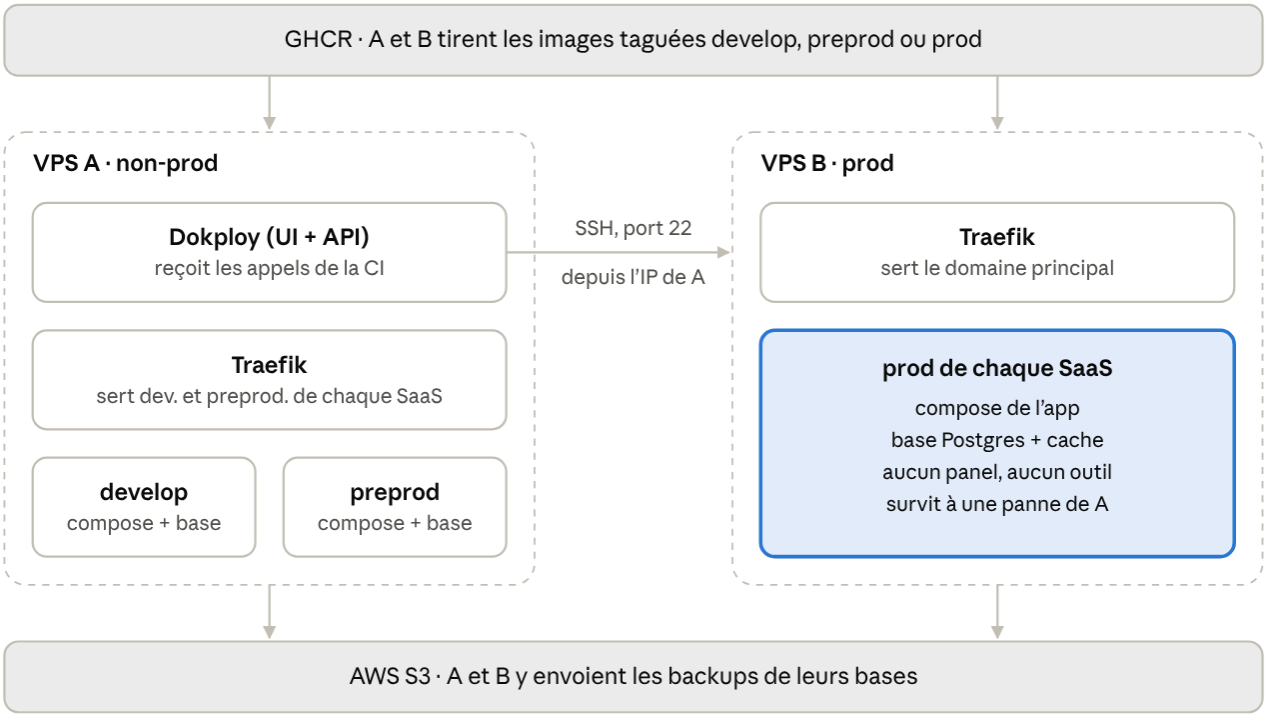
Ordre de mise en place

Étape	Quand	Résultat
1. Serveurs	une fois	A et B prêts, Dokploy pilote les deux
2. Sécurité	une fois	seuls 80/443 exposés au public
3. DNS	une fois, puis par SaaS (scripté)	Cloudflare gère tous les domaines
4 à 6. Monorepo, Dockerfiles, compose	une fois (template)	le modèle de tous tes SaaS
7. Dokploy	par SaaS (scripté)	1 projet, 3 environnements
8. CI/CD	une fois (repo platform)	build et promotion automatisés
9. Backups AWS	une fois, puis par SaaS (scripté)	bases et fichiers sur S3
10. Monitoring	une fois	tu es prévenu avant tes utilisateurs
11. Nouveau SaaS	à chaque idée	3 URLs en ligne

Étape 1 — Les deux serveurs

Le VPS A fait tourner Dokploy et tous les environnements non-prod ; le VPS B ne fait tourner que les prods, et Dokploy le pilote à distance par SSH.

Dokploy sur A pilote B par SSH ; B route seul le trafic de la prod



VPS A et VPS B · rôles et liaisons

La seule liaison entre les deux serveurs est le SSH de A vers B ; chacun tire ses images sur GHCR et envoie ses backups sur S3.

	VPS A (non-prod)	VPS B (prod)
Héberge	Dokploy (interface + API), Traefik, develop et preprod de tous les SaaS	Traefik, la prod de tous les SaaS
Bases	une par SaaS et par environnement	une par SaaS
Ouvert au public	80, 443	80, 443
SSH	depuis ton IP	depuis ton IP et l'IP de A
Charge	Dokploy + 2 stacks par SaaS	1 stack par SaaS
S'il tombe	plus de déploiements, la prod continue	prod coupée → plan de reprise (étape 9)

Dimensionnement. A porte deux stacks par SaaS plus Dokploy ; B n'en porte qu'une. A a donc souvent besoin d'autant de RAM que B, voire plus. Garde le même OS (Ubuntu LTS) et la même version de Docker sur les deux, pour que la preprod reste représentative de la prod.

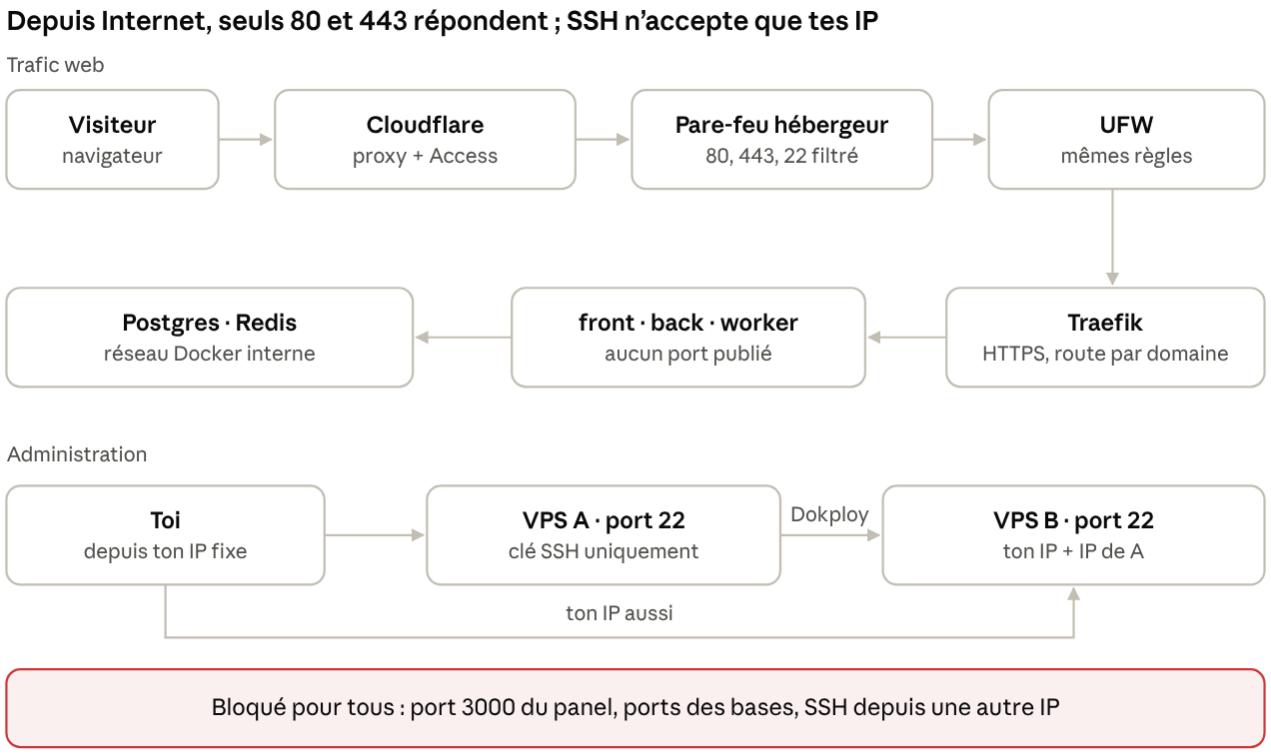
Pourquoi la prod survit à une panne de A. Sur un serveur distant, Dokploy n'installe qu'une instance Traefik (source) : B route lui-même son trafic. Si A tombe, tes utilisateurs ne voient rien ; tu ne peux simplement plus déployer avant d'avoir restauré A.

Mise en place (une seule fois)

1. Commande les deux VPS sous Ubuntu LTS et applique le durcissement de l'étape 2 sur chacun, avant toute installation. Dokploy cible Ubuntu et Debian LTS (source).
2. Sur A, installe Dokploy : `curl -sSL https://dokploy.com/install.sh | sh`. Ouvre `http://IP_A:3000` et crée le compte admin.
3. Dokploy → Web Server : assigne le domaine du panel (`dokploy.tonlab.fr`), certificat Let's Encrypt, HTTPS activé. Ferme ensuite l'accès direct au port 3000 (étape 2).
4. Active la 2FA ou une passkey sur ton compte.
5. Settings → SSH Keys : génère une clé et ajoute sa partie publique dans `/root/.ssh/authorized_keys` sur B.
6. Remote Servers → Add Server : IP de B, port SSH, utilisateur `root` , la clé créée. Puis Setup Server → Deployments pour installer Docker et Traefik sur B. L'accès root et bash comme shell par défaut sont obligatoires (source).
7. Onglet Security du serveur B : vérifie que les contrôles (UFW, SSH par clé, Fail2Ban) sont au vert (source).
8. Registry : ajoute GHCR (`ghcr.io` , ton utilisateur GitHub, un token classique limité à `read:packages`). Si le premier déploiement prod n'arrive pas à tirer les images, fais aussi un `docker login ghcr.io` en root sur B.
9. S3 Destinations : ajoute tes deux destinations AWS (étape 9).
10. Web Server → Backups : backup quotidien de Dokploy vers S3 (étape 9).

Étape 2 — Sécurité des serveurs

Une seule règle : depuis Internet, seuls les ports 80 et 443 répondent ; SSH n'accepte que des clés et des IP connues ; aucune base et aucun panel ne sont joignables en direct.



sécurité · trafic web, administration, flux bloqués

Une requête web traverse Cloudflare, le pare-feu de l'hébergeur, UFW puis Traefik ; les bases ne sont joignables que depuis les apps, sur le réseau Docker interne.

Les ports, serveur par serveur

Port	VPS A	VPS B	Autorisé depuis
22 (SSH)	ouvert	ouvert	A : ton IP · B : ton IP + IP de A
80 / 443	ouvert	ouvert	Internet (via Cloudflare)
3000 (panel Dokploy en direct)	fermé après le setup	absent	personne
5432, 6379… (bases)	jamais publié	jamais publié	réseau Docker interne uniquement

Durcissement commun à A et B

- [ ] Ubuntu LTS à jour, `unattended-upgrades` activé pour les correctifs de sécurité.
- [ ] SSH : `PasswordAuthentication no` et `PermitRootLogin prohibit-password` (Dokploy a besoin de root par clé sur B). Clés ed25519 uniquement.
- [ ] Fail2Ban (ou CrowdSec) sur SSH.

- [ ] UFW : `deny incoming` par défaut ; autoriser 22 depuis les IP listées, puis 80 et 443.
- [ ] Pare-feu réseau de l'hébergeur (si SSD Nodes en propose un dans son panel) avec les mêmes règles : c'est la seule barrière que Docker ne contourne pas.
- [ ] Rotation des logs Docker et purge des images (étape 10).
- [ ] Fuseau horaire et NTP corrects : les crons de backup en dépendent.

Le piège Docker / UFW

Docker écrit ses propres règles iptables : un port publié par un conteneur reste joignable même si UFW le bloque. Deux conséquences directes :

1. Ne publie jamais le port d'une base. Dans Dokploy, laisse vide le champ External Port des bases ; les apps les joignent par le réseau interne.
2. Le panel Dokploy écoute sur `0.0.0.0:3000` et UFW ne peut pas le bloquer (source). Une fois le domaine du panel en place, bloque 3000 au pare-feu de l'hébergeur. À défaut, retire la publication du port avec `docker service update --publish-rm "published=3000,target=3000,mode=host" dokploy` (source). Cette commande peut être annulée par une mise à jour : revérifie après chacune que `curl http://IP_A:3000` échoue depuis ton Mac.

Le panel Dokploy et son API

- Panel accessible uniquement via `https://dokploy.tonlab.fr`, avec 2FA ou passkey (source).
- Option renforcée : Cloudflare Access devant `dokploy.tonlab.fr` (connexion par email + code). GitHub Actions passe alors avec un service token Cloudflare dans ses en-têtes.
- Une clé API dédiée à GitHub Actions, stockée en secret GitHub, régénérée au moindre doute.

Les secrets : qui vit où

Secret	Où il vit	Qui l'utilise
Clé API Dokploy	secrets GitHub de chaque repo SaaS	workflows de déploiement
Token GHCR en lecture seule	registry Dokploy	A et B pour tirer les images
`DATABASE_URL`, clés d'app (Stripe, email…)	variables Dokploy, par environnement	conteneurs

Secret	Où il vit	Qui l'utilise
Clés AWS de backup	destinations S3 Dokploy	backups (étape 9)
Tokens Cloudflare, GitHub, AWS pour Terraform	gestionnaire de mots de passe	toi, au bootstrap
Clés SSH, codes de secours 2FA	gestionnaire de mots de passe	toi

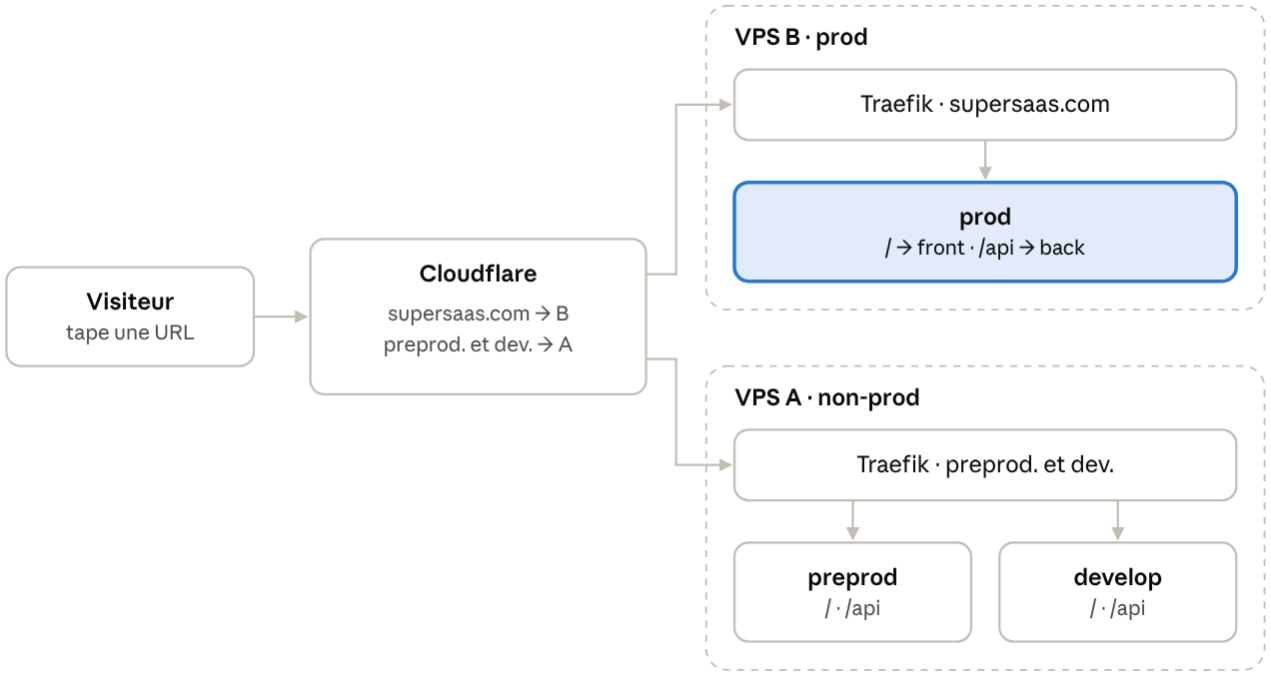
Règles : rien dans Git (seul `.env.example`, avec des valeurs factices) ; des secrets différents par environnement ; jamais le même mot de passe en prod et en preprod.

Protéger develop et preprod

- Cloudflare Access sur `dev.` et `preprod.` : seuls toi et tes testeurs passent, après un code reçu par email. Ajoute une règle de contournement pour `/api/health`, ou utilise le service token dans le smoke test de la CI.
- En-tête `X-Robots-Tag: noindex` sur ces deux environnements.
- Jamais de données de prod en develop ou preprod ; pour des tests réalistes, un dump anonymisé.

Étape 3 — DNS et domaines

Cloudflare gère le DNS de tous tes domaines : chaque SaaS reçoit les mêmes enregistrements, créés par Terraform, et Traefik obtient seul ses certificats.



DNS et routage · un domaine par environnement

Cloudflare choisit le serveur selon le nom demandé ; sur chaque serveur, Traefik envoie `/` au front et `/api` au back.

Une seule fois

1. Crée un compte Cloudflare (le plan gratuit suffit).
2. Pour chaque domaine acheté, `tonlab.fr` compris : ajoute-le dans Cloudflare, puis remplace ses serveurs DNS chez ton registrar par ceux que Cloudflare indique. C'est la seule étape manuelle par domaine.
3. SSL/TLS → mode Full (Strict) sur chaque domaine. Côté Dokploy, chaque domaine utilise Let's Encrypt avec HTTPS activé (source).
4. Respecte l'ordre : l'enregistrement DNS doit pointer vers le serveur avant que tu ajoutes le domaine dans Dokploy (source). Le bootstrap (étape 11) applique cet ordre pour toi.
5. Domaine plateforme : `dokploy.tonlab.fr` → IP de A, proxifié par Cloudflare.

Enregistrements par SaaS (créés par Terraform)

Nom	Type	Valeur	Proxy Cloudflare	Rôle
`supersaas.com`	A	IP de B	oui	prod

Nom	Type	Valeur	Proxy Cloudflare	Rôle
`www.supersaas.com`	CNAME	`supersaas.com`	oui	redirigé vers l'apex par une règle Cloudflare
`preprod.supersaas.com`	A	IP de A	oui	preprod, derrière Cloudflare Access
`dev.supersaas.com`	A	IP de A	oui	develop, derrière Cloudflare Access
`supersaas.com`	TXT	SPF de ton fournisseur d'email	non	autorise l'envoi d'emails
`<sélecteur>._domainkey`	CNAME ou TXT	DKIM fourni par le fournisseur	non	signe tes emails
`_dmarc.supersaas.com`	TXT	`v=DMARC1; p=none; rua=mailto:dmarc@supersaas.com`, puis `p=quarantine`	non	politique anti-usurpation

Pour les emails transactionnels (inscription, mot de passe oublié), Amazon SES est cohérent avec ton compte AWS ; Resend ou Postmark marchent aussi. Chacun te donne ses valeurs SPF et DKIM exactes.

Pourquoi des noms explicites plutôt qu'un wildcard

Terraform crée ces enregistrements sans effort, donc le wildcard n'apporte plus rien. Les noms explicites séparent proprement A et B, n'envoient aucun sous-domaine inventé vers A, et permettent une règle Cloudflare Access par nom.

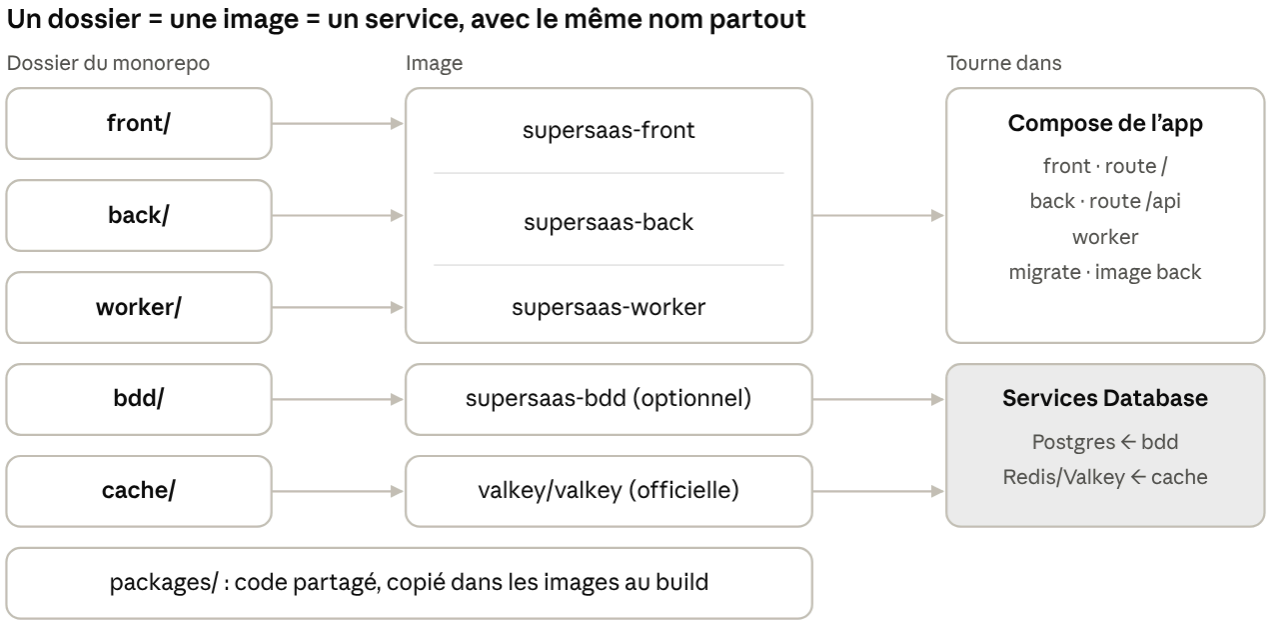
Garde aussi des noms à un seul niveau : avec le proxy Cloudflare, `api.exemple.com` fonctionne mais pas `staging.api.exemple.com` (source).

Le routage dans chaque environnement

Un seul domaine par environnement : `/` va au front, `/api` va au back. Conséquences : pas de CORS, des cookies d'authentification simples, et un front qui appelle l'API en relatif. C'est aussi ce qui permet de promouvoir la même image d'un environnement à l'autre (étape 5).

Étape 4 — Structure du monorepo

Chaque brique a son dossier et son Dockerfile ; le nom du dossier devient le nom de l'image, du service compose et du service Dokploy, ce qui permet à la CI de tout découvrir seule.



monorepo · dossier, image, lieu d'exécution

Les briques applicatives finissent dans le compose de l'app ; `bdd` et `cache` deviennent des services Database, séparés des déploiements.

```
supersaas/
├── saas.yaml              # manifeste : nom, domaine, briques à builder
├── front/                 # Dockerfile → ghcr.io/toi/supersaas-front
├── back/                  # Dockerfile → ghcr.io/toi/supersaas-back
├── bdd/
│   ├── Dockerfile         # optionnel : FROM postgres + extensions
│   ├── init/              # extensions, jouées au tout premier démarrage
│   ├── migrations/        # SQL versionné, joué par le job migrate
│   └── seed/              # données de démo pour develop
├── worker/                # Dockerfile (jobs asynchrones, optionnel)
├── cache/                 # pas de Dockerfile : config de Redis/Valkey
├── packages/              # code partagé : types, validation, config lint
├── infra/
│   ├── compose.yml        # local : tout le stack sur ton Mac
│   ├── compose.deploy.yml # Dokploy : front, back, worker, migrate
│   └── .env.example       # toutes les variables, valeurs factices
├── .github/workflows/     # 3 petits fichiers qui appellent toi/platform@v1
└── Makefile               # make dev, make test, make db-migrate, make db-
reset
```

Le rôle de chaque dossier

Dossier	Dockerfile	Image produite	Où il tourne une fois déployé
`front/`	oui	`supersaas-front`	compose de l'app, route `/`
`back/`	oui	`supersaas-back`	compose de l'app, route `/api` ; sert aussi au job `migrate`
`worker/`	oui	`supersaas-worker`	compose de l'app, sans route
`bdd/`	optionnel	`supersaas-bdd` (si extensions)	service Database Dokploy, hors du compose
`cache/`	non	image officielle Redis/Valkey	service Database Dokploy
`packages/`	non	aucune	copié dans les images qui l'utilisent

Les migrations vivent dans `bdd/migrations/`, mais c'est l'image du back qui les exécute : son Dockerfile les copie, et le job `migrate` lance l'outil de migration avant le démarrage du back.

`saas.yaml`, le manifeste (format proposé)

```
name: supersaas
domain: supersaas.com
build: [front, back, worker]       # dossiers avec un Dockerfile applicatif
database: { dockerfile: bdd }      # ou { image: postgres:17 } sans
Dockerfile
cache: { image: valkey/valkey:8 }
routes: { front: "/", back: "/api" }
```

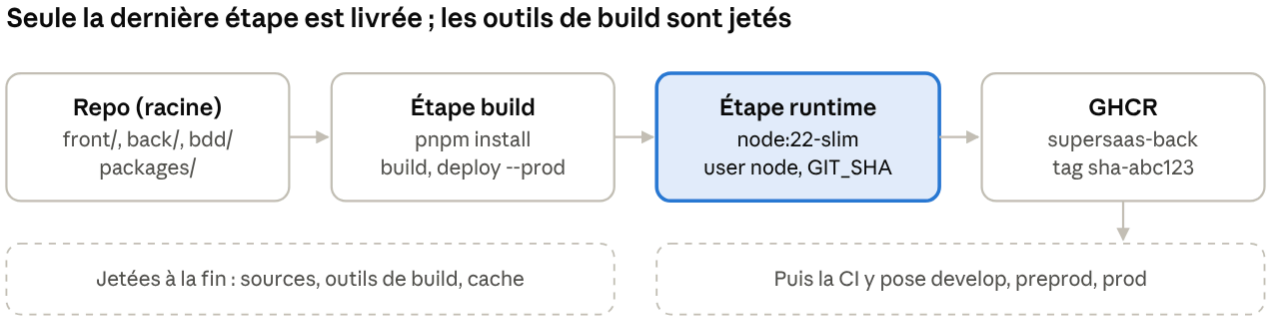
Le bootstrap (étape 11) et les workflows (étape 8) lisent ce fichier. Ajouter une brique = créer le dossier avec son Dockerfile et l'ajouter à `build`.

Outillage

En TypeScript : pnpm workspaces + Turborepo. Turborepo sait ne lancer les tests que sur ce qui a changé (`--affected`) et extraire le sous-ensemble du repo utile à un Dockerfile (`turbo prune`). En Python, l'équivalent est un workspace `uv`.

Étape 5 — Dockerfiles

Tous les Dockerfiles applicatifs suivent le même gabarit multi-stage : compilation dans des étapes jetables, image finale minimale, utilisateur non-root, version exposée sur `/api/health`.



Dockerfile multi-stage · du repo à GHCR

L'image poussée ne contient que l'app compilée et son runtime ; c'est elle, et seulement elle, qui voyage ensuite de develop à prod.

Les règles du gabarit

- **Contexte de build = racine du repo** (`docker build -f back/Dockerfile .`). Sinon `back/` ne voit ni `packages/` ni `bdd/migrations/`.

- **Image de base épinglée** (`node:22-slim`, jamais `latest`). Défaut : `-slim`. Alpine seulement si tout fonctionne avec `musl` : Prisma, `sharp` et des dépendances Python natives y posent souvent problème. Distroless pour du Go.
- **Utilisateur non-root** (`USER node` ou un utilisateur créé).
- **`ARG GIT_SHA`** fourni par la CI et renvoyé par `/api/health` : c'est ce que le smoke test vérifie après chaque déploiement.
- **Label `org.opencontainers.image.source`** = URL du repo : GHCR rattache l'image au repo et à ses droits d'accès.
- **`.dockerignore`** à la racine : `node_modules`, `.git`, `.env*`.
- **Aucun secret dans l'image** : tout arrive par variables d'environnement au démarrage.

Le piège des variables figées au build

La même image part en develop, en preprod puis en prod. Une variable figée au build (par exemple `NEXT_PUBLIC_API_URL` avec Next.js) garderait donc sa valeur de develop jusqu'en prod. Règle : aucune valeur propre à un environnement au moment du build. Le front appelle l'API en relatif (`/api`), et le reste se lit au démarrage du conteneur.

Exemple : `back/Dockerfile` (Node + pnpm, à adapter)

```
# Étape jetable : dépendances + build
FROM node:22-slim AS build
WORKDIR /repo
RUN corepack enable
COPY . .
RUN pnpm install --frozen-lockfile \
 && pnpm --filter back build \
 && pnpm --filter back deploy --prod /out   # pnpm 10 : inject-workspace-
packages=true dans .npmrc

# Image livrée
FROM node:22-slim AS runtime
ARG GIT_SHA=dev
ENV NODE_ENV=production GIT_SHA=$GIT_SHA
LABEL org.opencontainers.image.source="https://github.com/toi/supersaas"
WORKDIR /app
COPY --from=build --chown=node:node /out ./
COPY --from=build --chown=node:node /repo/bdd/migrations ./migrations
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

Le `front/Dockerfile` suit le même modèle. Avec Next.js, active `output: "standalone"` pour une image finale légère.

`bdd/Dockerfile` (seulement si tu as besoin d'extensions)

```
FROM postgres:17
RUN apt-get update \
 && apt-get install -y --no-install-recommends postgresql-17-pgvector \
 && rm -rf /var/lib/apt/lists/*
COPY init/ /docker-entrypoint-initdb.d/
```

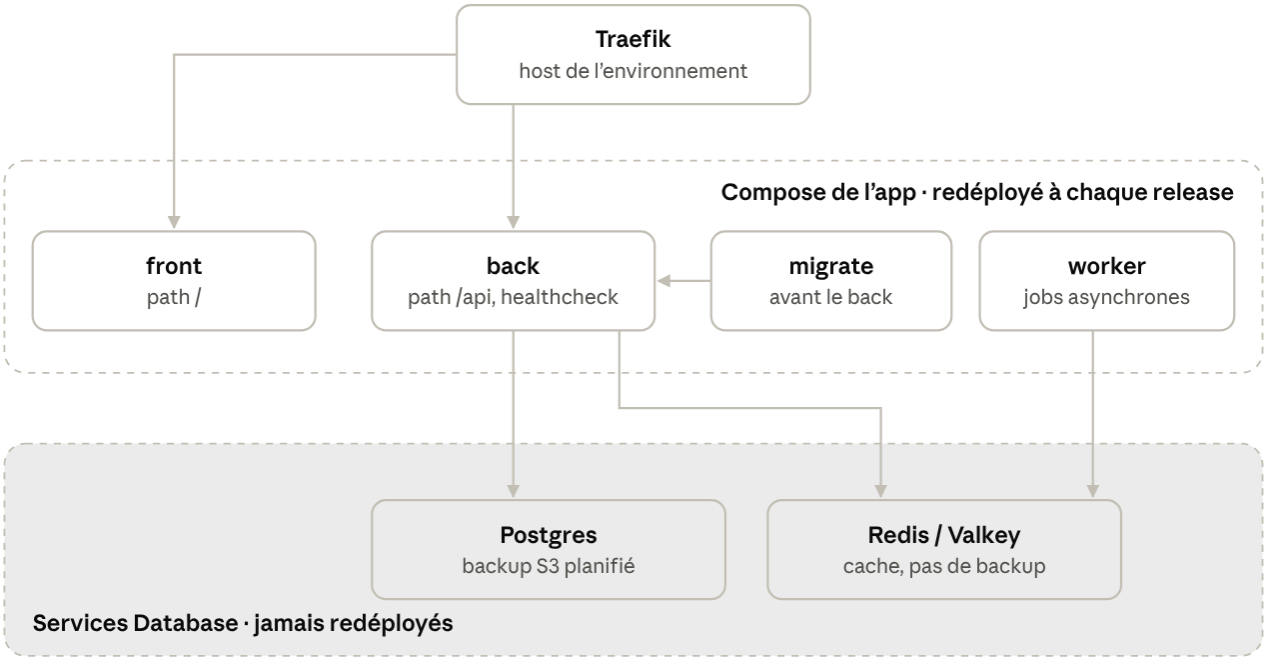
Les scripts de `bdd/init/` ne s'exécutent qu'au premier démarrage, sur un volume vide. Tout changement de schéma ultérieur passe par `bdd/migrations/`. Sans besoin d'extension, pas de Dockerfile : l'image officielle `postgres:17` suffit.

L'image `bdd` n'est pas promue comme les autres : elle porte une version explicite (`supersaas-bdd:pg17-1`), car changer l'image d'une base est une opération volontaire, testée en develop puis preprod avant la prod.

Étape 6 — Docker compose : local et déploiement

Deux fichiers : `compose.yml` lance tout le stack sur ton Mac ; `compose.deploy.yml`, identique pour les trois environnements, ne contient que les services sans état.

Le compose de l'app est redéployé à chaque release ; les bases jamais



un environnement déployé · app et données séparées

Traefik ne parle qu'au front et au back ; migrate passe avant le back, et seuls back et worker touchent aux données.

Qui tourne où

Service	En local (`compose.yml`)	Sur Dokploy
front	build depuis `front/` , rechargement à chaud	`compose.deploy.yml` , image GHCR
back	build depuis `back/` , rechargement à chaud	`compose.deploy.yml` , image GHCR
worker	build depuis `worker/`	`compose.deploy.yml` , image GHCR
migrate	`make db-migrate`	`compose.deploy.yml` , image back, avant le back
bdd	build depuis `bdd/` , port `127.0.0.1:5432`	service Database Postgres, hors compose
cache	image officielle, port `127.0.0.1:6379`	service Database Redis, hors compose

En local, publie les ports sur `127.0.0.1` uniquement. Sur Mac, OrbStack est une alternative plus légère à Docker Desktop.

Pourquoi séparer l'app et les données

Le compose de l'app est redéployé à chaque release ; les bases ne le sont jamais. Les services Database de Dokploy ont des backups planifiés intégrés (étape 9) et acceptent une image personnalisée comme ton image `bdd` (source). Les apps les joignent par leur nom interne, par exemple `supersaas-prod-db:5432` .

`depends_on` ne suffit pas

`depends_on` seul attend que le conteneur démarre, pas qu'il soit prêt. On combine donc trois mécanismes :

1. Un job `migrate` éphémère ; le back attend `condition: service_completed_successfully` .
2. Un `healthcheck` sur le back.
3. Le front attend `condition: service_healthy` sur le back.

Si une migration échoue, le back ne démarre pas, le déploiement est marqué en échec et le smoke test de la CI te prévient.

`infra/compose.deploy.yml`

```
x-app: &app
  pull_policy: always            # retire l'image à chaque déploiement
  restart: unless-stopped
  networks: [dokploy-network]    # pour joindre les bases Dokploy
  logging:
    driver: json-file
    options: { max-size: "10m", max-file: "3" }

services:
  migrate:
    <<: *app
    image: ghcr.io/toi/supersaas-back:${DEPLOY_ENV}
    command: ["node", "dist/migrate.js"]
    restart: "no"
    environment:
      DATABASE_URL: ${DATABASE_URL}

  back:
    <<: *app
    image: ghcr.io/toi/supersaas-back:${DEPLOY_ENV}
    environment:
      DATABASE_URL: ${DATABASE_URL}
      REDIS_URL: ${REDIS_URL}
      APP_SECRET: ${APP_SECRET}
    depends_on:
      migrate: { condition: service_completed_successfully }
    healthcheck:
      test: ["CMD", "node", "dist/healthcheck.js"]
      interval: 10s
      retries: 6
    deploy:
      resources:
        limits: { cpus: "${BACK_CPUS:-2}", memory: "${BACK_MEMORY:-1g}" }

  front:
    <<: *app
    image: ghcr.io/toi/supersaas-front:${DEPLOY_ENV}
    depends_on:
      back: { condition: service_healthy }
    deploy:
      resources:
        limits: { cpus: "${FRONT_CPUS:-1}", memory: "${FRONT_MEMORY:-512m}" }

  worker:
```

```
    <<: *app
    image: ghcr.io/toi/supersaas-worker:${DEPLOY_ENV}
    environment:
      DATABASE_URL: ${DATABASE_URL}
      REDIS_URL: ${REDIS_URL}
    depends_on:
      migrate: { condition: service_completed_successfully }

networks:
  dokploy-network:
    external: true
```

Ce qu'il faut retenir :

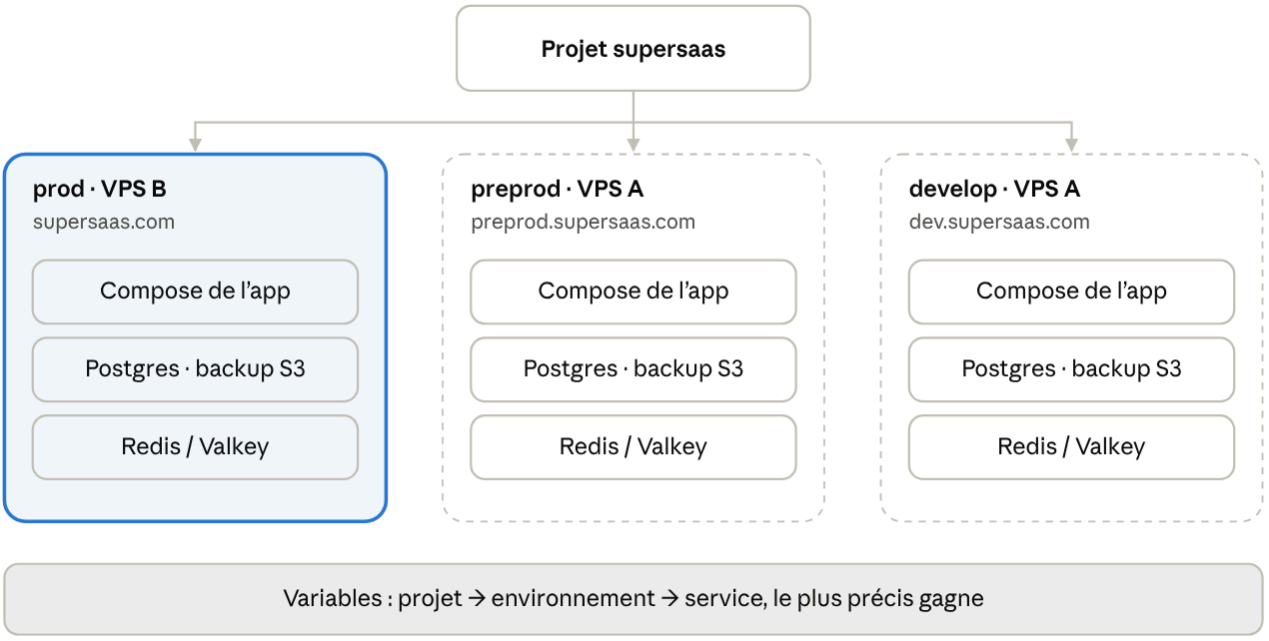
- `DEPLOY_ENV` vaut `develop`, `preprod` ou `prod` : c'est l'étiquette d'image que la CI déplace (étape 8).
- Aucun `ports:` : Traefik route par les domaines Dokploy, et rien n'est exposé en direct.
- Les plafonds CPU/RAM ont des valeurs par défaut généreuses pour la prod ; sur A, develop et preprod définissent des valeurs plus basses dans leurs variables.
- Les valeurs de mémoire et les commandes (`dist/migrate.js`, `dist/healthcheck.js`) sont des exemples à adapter à ton back.

Étape 7 — Dokploy : un projet par SaaS

Chaque SaaS = un projet Dokploy avec trois environnements ; develop et preprod tournent sur A, prod sur B, et chaque environnement a sa propre base, son compose et son domaine.

Dokploy organise tout en Organisation → Projet → Environnement → Service, et chaque environnement isole ses services des autres (source).

Un projet Dokploy par SaaS, trois environnements, chacun sur son serveur



projet Dokploy · 3 environnements, 3 services chacun

Les trois environnements ont exactement la même forme ; seuls le serveur, le domaine et les variables changent.

Contenu d'un environnement

Environnement	Serveur	Services	Domaine	Déployé par
prod	B	base, cache, compose de l'app	`supersaas.com`	release GitHub
preprod	A	base, cache, compose de l'app	`preprod.supersaas.com`	bouton Promote
develop	A	base, cache, compose de l'app	`dev.supersaas.com`	merge sur main (auto)

Réglages du service compose

- **Serveur** : A pour develop et preprod, B pour prod, choisi à la création.
- **Source « Raw »** : la CI y pousse le contenu de `compose.deploy.yml` du sha promu (`compose.update`), puis déclenche `compose.deploy` (source). Le fichier compose et les images ont ainsi toujours la même version. Vérifie le nom exact des champs dans la doc API de ta version.

- **Déploiement automatique sur push Git** : désactivé. Seule la CI déclenche les déploiements.
- **Domaines** (onglet Domains) : `front` → host de l'environnement, path `/`, port 3000 ; `back` → même host, path `/api`, port de l'API. HTTPS avec Let's Encrypt. Le back sert ses routes sous le préfixe `/api`.
- **Tout changement de domaine exige un redéploiement** du compose (source).

Les variables, trois niveaux

Dokploy applique les variables du projet, puis celles de l'environnement, puis celles du service : le niveau le plus précis gagne (source).

Niveau	Exemples	Pourquoi à ce niveau
Projet	`APP_NAME`, URL d'un service tiers commun	identique partout
Environnement	`DEPLOY_ENV=preprod`, `BACK_MEMORY=512m`	change d'un environnement à l'autre
Service	`DATABASE_URL`, `APP_SECRET`, clés Stripe test ou live	secrets propres à l'environnement

Les services Database

- **Postgres** : image `ghcr.io/toi/supersaas-bdd:pg17-1` si tu as un `bdd/Dockerfile`, sinon `postgres:17`. Nom interne explicite (`supersaas-prod-db`). External Port vide. Backup planifié vers S3 (étape 9).
- **Redis / Valkey** : nom interne `supersaas-prod-cache`, External Port vide. Utilisé comme cache, il n'est pas sauvegardé. S'il porte une file de jobs que tu ne peux pas perdre, active sa persistance et sauvegarde son volume (étape 9).
- **Une base par environnement**, jamais partagée : tu peux tester une montée de version de Postgres en develop, puis preprod, avant la prod.

Étape 8 — CI/CD : build une fois, promouvoir ensuite

Un merge sur `main` construit les images une seule fois et les déploie en develop ; un bouton les promeut en preprod ; une release GitHub les promeut en prod. Aucun rebuild entre les environnements.

pipeline · build unique, promotion en 3 environnements

Le build n'a lieu qu'une fois, au merge ; tout ce qui suit déplace des étiquettes sur la même image.

Les branches

- Une seule branche longue : `main`, avec PR et CI verte obligatoires (sur un repo privé, la protection de branche demande GitHub Pro).
- Des branches de feature courtes (`feat/…`), mergées en squash.
- Pas de branches develop, preprod ou prod : un environnement est une étiquette d'image, pas une branche.

Les workflows réutilisables (repo `toi/platform`)

Fichier	Déclenché par	Ce qu'il fait
`ci.yml`	chaque PR	lint, tests, build à blanc des briques modifiées
`build.yml`	push sur `main`	build + push `ghcr.io/toi/<saas>-<brique>:sha-<sha>` pour chaque brique de `saas.yaml`
`deploy.yml`	build, bouton ou release	étiquette d'environnement, compose poussé, appel Dokploy, smoke test

Versionne-les par tag (`@v1`). Comme `platform` est privé : Settings → Actions → General → Access, autorise les repos de ton compte.

Les 3 fichiers d'un SaaS

```
# .github/workflows/ci.yml
on: pull_request
jobs:
  ci:
    uses: toi/platform/.github/workflows/ci.yml@v1
```

```
# .github/workflows/release.yml — merge sur main → develop
on:
  push: { branches: [main] }
jobs:
  build:
    uses: toi/platform/.github/workflows/build.yml@v1
    secrets: inherit
  develop:
    needs: build
    uses: toi/platform/.github/workflows/deploy.yml@v1
    with: { env: develop, sha: "${{ github.sha }}" }
    secrets: inherit
```

```
# .github/workflows/promote.yml — bouton → preprod, release → prod
on:
  workflow_dispatch:
    inputs:
      sha: { description: "sha à promouvoir (vide = dernier commit de main)",
required: false }
  release: { types: [published] }
jobs:
  promote:
    uses: toi/platform/.github/workflows/deploy.yml@v1
    with:
      env: ${{ github.event_name == 'release' && 'prod' || 'preprod' }}
      sha: ${{ inputs.sha || github.sha }}
    secrets: inherit
```

Ce que fait `deploy.yml`, dans l'ordre

1. Se connecte à GHCR.
2. Vérifie que l'image `sha-<sha>` existe pour chaque brique ; sinon il s'arrête, rien n'est déployé à moitié.
3. Pose l'étiquette d'environnement sur ces images, sans rebuild.
4. Pousse dans Dokploy le `compose.deploy.yml` de ce sha, puis déclenche le déploiement.
5. Smoke test : interroge `https://<host>/api/health` jusqu'à lire le bon sha (délai maximum de quelques minutes), sinon échec.
6. Enregistre un deployment GitHub : l'historique de ce qui tourne où reste visible dans le repo.

```
# extrait de deploy.yml (simplifié)
- name: Promote images
  run: |
    for app in $APPS; do
      docker buildx imagetools create \
        -t ghcr.io/toi/$SAAS-$app:${{ inputs.env }} \
        ghcr.io/toi/$SAAS-$app:sha-${{ inputs.sha }}
    done
- name: Deploy on Dokploy
  run: |
    curl -fsS -X POST "$DOKPLOY_URL/api/compose.deploy" \
      -H "x-api-key: ${{ secrets.DOKPLOY_API_KEY }}" \
      -H "Content-Type: application/json" \
      -d "{\"composeId\": \"$COMPOSE_ID\"}"
- name: Smoke test
  run: ./scripts/wait-for-version.sh "https://$HOST/api/health" "${{
inputs.sha }}"
```

Variables du repo SaaS, écrites par le bootstrap : `COMPOSE_ID_DEVELOP`, `COMPOSE_ID_PREPROD`, `COMPOSE_ID_PROD` et les trois hosts. Secrets : `DOKPLOY_URL`, `DOKPLOY_API_KEY`, plus le service token Cloudflare si tu protèges le panel.

Rollback

Relance `promote` avec le sha précédent : l'étiquette `prod` revient sur l'ancienne image et Dokploy redéploie. Condition : des migrations rétrocompatibles (ajouter d'abord, migrer, supprimer seulement dans une release suivante), pour que l'ancienne version fonctionne avec le schéma actuel.

Ce que les plans GitHub imposent

Sur les plans Free, Pro et Team, les approbateurs obligatoires d'un environnement ne sont disponibles que pour les repos publics ; et les environnements d'un repo privé demandent au moins GitHub Pro (source). D'où ce modèle : la validation manuelle, c'est le bouton et la release ; variables et secrets vivent au niveau du repo.

Étape 9 — Backups sur AWS S3

Tout ce qui ne peut pas être reconstruit depuis Git part automatiquement vers S3 en région Paris (`eu-west-3`), dans des buckets séparés prod et non-prod, versionnés et protégés contre la suppression ; une restauration est testée chaque mois.

backups AWS · qui écrit, avec quel accès, dans quel bucket

Un serveur compromis ne peut toucher qu'à son propre bucket, et le bucket prod refuse toute suppression pendant 30 jours.

Quoi sauvegarder, et comment

Fréquences et rétentions ci-dessous = propositions de départ, à ajuster selon ce que tu acceptes de perdre.

Élément	Où il vit	Méthode	Fréquence	Rétention
Bases prod	VPS B	backup de base Dokploy → `tonlab-backups-prod`	1 fois par jour, la nuit	30 jours
Bases develop et preprod	VPS A	backup de base Dokploy → `tonlab-backups-nonprod`	1 fois par jour	7 jours
Config Dokploy (projets, domaines, certificats)	VPS A	backup Web Server Dokploy → `tonlab-backups-nonprod`	1 fois par jour	30 jours
Fichiers des utilisateurs	S3, directement	l'app écrit dans `supersaas-prod-files`	en continu	versions supprimées gardées 30 jours
Volumes Docker nommés (si une brique en a)	A ou B	volume backup Dokploy → bucket de l'environnement	1 fois par jour	14 jours
État Terraform	S3	backend S3 avec verrou natif	à chaque `apply`	versioning
Cache Redis	A, B	aucune : il se reconstruit	—	—
Code, Dockerfiles, workflows	GitHub	aucune action ; les images se reconstruisent	—	—

Élément	Où il vit	Méthode	Fréquence	Rétention
Secrets	gestionnaire de mots de passe	export chiffré du coffre	à chaque changement	—

Ce que fait Dokploy, précisément :

- **Backup de base** : tu choisis la destination S3, la base, un cron et un préfixe ; un bouton Test envoie un premier backup pour vérifier (source). Utilise le préfixe `<saas>/<env>/`.
- **Backup Web Server** : sauvegarde la base interne de Dokploy et `/etc/dokploy` dans un seul zip, restaurable en un clic (source). Il ne contient pas les données de tes apps.
- **Volume backup** : uniquement pour les volumes Docker nommés, pas pour les montages de dossiers (source).

Les fichiers des utilisateurs vont directement sur S3

Avatars, PDF, imports : l'app les écrit dans un bucket S3 et les sert par URL signée. Rien sur le disque des VPS, donc rien à sauvegarder côté serveur et rien à migrer si un VPS change.

Les buckets (créés par Terraform)

Bucket	Contenu	Réglages
`tonlab-backups-prod`	dumps des bases prod	versioning, Object Lock en mode Governance (30 jours), versions non courantes supprimées après 30 jours
`tonlab-backups-nonprod`	dumps develop/preprod, backups Dokploy	versioning, versions non courantes supprimées après 7 jours
`<saas>-prod-files`	fichiers des utilisateurs prod	versioning, accès par URL signées
`<saas>-nonprod-files`	fichiers develop/preprod	versioning
`tonlab-tfstate`	état Terraform	versioning

Sur tous : Block Public Access activé, chiffrement par défaut, région `eu-west-3`. Object Lock s'active le plus simplement à la création du bucket. Son rôle : même si B est compromis et ses clés volées, les backups prod des 30 derniers jours ne peuvent pas être effacés.

Les accès IAM : un utilisateur par usage, droits minimaux

Utilisateur IAM	Utilisé par	Peut	Ne peut pas
`dokploy-backup-prod`	destination S3 « aws-prod » dans Dokploy	lister, lire, écrire, supprimer dans `tonlab-backups-prod`	toucher aux autres buckets ou aux réglages
`dokploy-backup-nonprod`	destination S3 « aws-nonprod » dans Dokploy	idem dans `tonlab-backups-nonprod`	voir la prod
`app-supersaas-prod`	back et worker prod	lire et écrire dans `supersaas-prod-files`	tout le reste
`app-supersaas-nonprod`	back et worker develop/preprod	lire et écrire dans `supersaas-nonprod-files`	tout le reste
ton compte admin (MFA)	Terraform, console	tout	—

Compte racine AWS : MFA activée, aucune clé d'accès, jamais utilisé au quotidien. Ajoute une alerte AWS Budgets sur un montant mensuel qui te va.

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Effect": "Allow", "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3:::tonlab-backups-prod" },
    { "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject",
"s3:DeleteObject"],
      "Resource": "arn:aws:s3:::tonlab-backups-prod/*" }
  ]
}
```

Dans Dokploy, une destination AWS = clé d'accès + secret de l'utilisateur IAM, bucket, région `eu-west-3`, endpoint `https://s3.eu-west-3.amazonaws.com`.

Test de restauration mensuel

1. Sur A, crée un environnement temporaire `restore-test` avec une base Postgres vide.
2. Restaure dedans le dernier backup prod depuis la destination « aws-prod ». Jamais dans preprod : ce sont des données personnelles réelles.

3. Vérifie le nombre de lignes des tables clés et la date de la dernière écriture.
4. Supprime l'environnement temporaire et note la date et la durée de l'opération.

Plan de reprise si B est perdu (prod coupée)

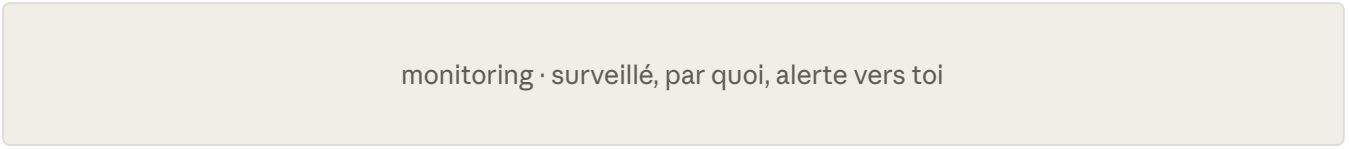
1. Commande un nouveau VPS et applique l'étape 2.
2. Ajoute-le dans Dokploy comme serveur distant, puis Setup Server.
3. Relance le bootstrap pour l'environnement prod de chaque SaaS, sur ce serveur (étape 11).
4. Restaure le dernier backup de chaque base prod.
5. Relance `promote` avec le sha de prod en cours.
6. Mets la nouvelle IP dans Terraform (`ip_prod`) et lance `terraform apply` : le DNS bascule.

Plan de reprise si A est perdu (la prod continue)

1. Nouveau VPS, étape 2, installation de Dokploy.
2. Web Server → Backups → Restore depuis le dernier backup Dokploy ; si l'IP a changé, mets-la à jour dans Web Server → Server (source).
3. Mets la nouvelle IP de A dans Terraform (`dokploy.` , `dev.` , `preprod.`) et dans la règle SSH de B.
4. Restaure les bases develop et preprod, ou repars des seeds.

Étape 10 — Monitoring et exploitation

Tu dois apprendre une panne avant tes utilisateurs : une sonde externe sur chaque URL, les erreurs applicatives centralisées, le disque surveillé et l'entretien courant automatisé.



Chaque chose surveillée a son outil, et toutes les alertes convergent vers un seul canal : le tien.

Ce qui est surveillé

Quoi	Outil	Alerte quand
`/api/health` des 3 environnements + panel Dokploy	sonde externe (UptimeRobot, Better Stack…)	2 échecs consécutifs
Expiration des certificats	la même sonde	moins de 14 jours
CPU, RAM, disque de B	agent léger sur B, ou alertes de l'hébergeur	disque > 80 %
CPU, RAM, disque de A	monitoring intégré de Dokploy	disque > 80 %
Erreurs applicatives	Sentry, ou GlitchTip auto-hébergé sur A	nouvelle erreur en prod
Backups et déploiements	notifications Dokploy (email, Telegram, Slack…)	un backup ou un déploiement échoue
Coûts AWS	AWS Budgets	dépassement du budget mensuel

Pourquoi un agent sur B : Dokploy ne fait pas le monitoring des serveurs distants (source). Et la sonde doit vivre hors de tes serveurs : hébergée sur A, elle tomberait avec lui.

Entretien automatique, sur A et B

- **Rotation des logs Docker**, avant de lancer le moindre conteneur : dans `/etc/docker/daemon.json`, mets `{"log-driver": "json-file", "log-opts": {"max-size": "10m", "max-file": "3"}}`, puis `systemctl restart docker`. Plafonne aussi le journal système (`SystemMaxUse=200M` dans `/etc/systemd/journald.conf`).
- **Purge des images**, chaque nuit en cron : `docker image prune -af --filter "until=168h" && docker builder prune -af --filter "until=168h"`. Les images de moins de 7 jours restent sur place pour un rollback rapide ; les plus anciennes se re-téléchargent depuis GHCR.
- **Attention au nettoyage quotidien intégré de Dokploy** : il supprime aussi les volumes Docker non utilisés (source). Une base arrêtée pourrait perdre son volume. Préfère le cron ci-dessus, ou n'arrête jamais une base tant que ce nettoyage est actif.
- **Correctifs système** : `unattended-upgrades` (étape 2), redémarrage planifié hors heures de pointe quand le noyau l'exige.

- **Dépendances** : Dependabot sur chaque repo (écosystèmes `docker`, `npm` ou `pip`, `github-actions`). Ses PR passent par la CI comme les tiennes.

Mise à jour de Dokploy

Une fois par mois, après lecture des notes de version : `curl -sSL https://dokploy.com/install.sh | sh -s update` sur A (source). Les notes indiquent s'il faut relancer le Setup Server de B (source). Fais un backup Web Server juste avant, puis revérifie que le port 3000 reste fermé.

Étape 11 — Lancer un nouveau SaaS en un éclair

Plateforme en place, une idée devient trois URLs en ligne en quatre gestes : acheter le domaine, pousser ton monorepo, remplir `saas.yaml`, lancer `make new-saas`.



Tu fais les quatre gestes du haut ; tout le reste est créé par la commande, puis par le pipeline.

Les quatre gestes

1. Achète `supersaas.com` et pointe ses serveurs DNS vers Cloudflare (seule étape manuelle).
2. Crée ton monorepo (depuis `toi/saas-template`, ou à la main en suivant l'étape 4) et pousse-le sur GitHub.
3. Remplis `saas.yaml` : nom, domaine, briques à builder.
4. Depuis le repo `platform` : `make new-saas SAAS=supersaas`.

Ce que fait `make new-saas`

Ordre	Outil	Crée
1	Terraform, provider Cloudflare	enregistrements DNS de l'étape 3, règles Access sur dev et preprod
2	Terraform, provider AWS	buckets de fichiers et utilisateurs IAM de l'app (étape 9)
3	Terraform, provider GitHub	secrets Actions, réglages du repo (squash merge, suppression des branches mergées)

Ordre	Outil	Crée
4	script contre l'API Dokploy	projet, 3 environnements, bases (A, A, B) avec backups, services compose, domaines, variables
5	script	`composeId` et hosts écrits dans les variables du repo GitHub
6	GitHub Actions	build + develop, puis promotion en preprod, puis release `v0.1.0` en prod

Le script Dokploy est idempotent : il vérifie l'existence de chaque élément avant de le créer. Tu peux donc le relancer sans doublon, et il sert aussi au plan de reprise (étape 9).

Pourquoi Terraform d'un côté et un script de l'autre

Cloudflare, AWS et GitHub ont des providers Terraform officiels et matures ; c'est aussi la compétence la plus utile en entreprise. Pour Dokploy, le seul outil « as code » trouvé est un provider Pulumi communautaire en version 0.3.1 (source) : trop jeune pour en dépendre, l'API suffit.

Le repo `platform`

```
platform/
├── .github/workflows/      # ci.yml, build.yml, deploy.yml (réutilisables,
tag v1)
├── terraform/
│   ├── base/               # buckets de backup + tfstate, IAM de backup, DNS
de tonlab.fr
│   ├── modules/saas/       # DNS + buckets + IAM + réglages GitHub d'un SaaS
│   └── saas/supersaas.tf   # un fichier par SaaS
├── scripts/
│   ├── dokploy-bootstrap.ts
│   └── wait-for-version.sh
└── Makefile                # make new-saas, make plan, make apply
```

```
# terraform/saas/supersaas.tf
module "supersaas" {
  source     = "../modules/saas"
  name       = "supersaas"
  domain     = "supersaas.com"
  ip_nonprod = var.ip_vps_a
  ip_prod    = var.ip_vps_b
}
```

```
# backend : l'état vit sur S3, verrou natif sans DynamoDB
terraform {
  backend "s3" {
    bucket       = "tonlab-tfstate"
    key          = "platform/terraform.tfstate"
    region       = "eu-west-3"
    use_lockfile = true
  }
}
```

Le verrou natif S3 (`use_lockfile`) existe depuis Terraform 1.10, et le verrou DynamoDB est déprécié (source).

Ensuite, au quotidien

Une branche de feature, une PR, un merge : c'est en develop. Un clic sur Promote : c'est en preprod. Une release : c'est en prod.

Checklists

Mise en place unique de la plateforme

- [ ] AWS : MFA sur le compte racine, compte admin avec MFA, alerte AWS Budgets
- [ ] Terraform `base` appliqué : buckets de backup et d'état, utilisateurs IAM de backup, DNS de `tonlab.fr`
- [ ] VPS A et B durcis (étape 2), même version d'Ubuntu et de Docker
- [ ] `daemon.json` et cron de purge en place sur A et B
- [ ] Dokploy installé sur A, domaine du panel en HTTPS, 2FA, port 3000 fermé
- [ ] B ajouté en serveur distant, onglet Security au vert
- [ ] Dans Dokploy : registry GHCR, destinations « aws-prod » et « aws-nonprod », backup Web Server quotidien
- [ ] Repo `platform` (workflows `@v1` , Terraform, scripts) et repo `saas-template`
- [ ] Sonde externe, Sentry et notifications Dokploy branchées
- [ ] Premier test de restauration réussi

À chaque nouveau SaaS

- [ ] Domaine acheté, serveurs DNS sur Cloudflare
- [ ] Monorepo poussé, `saas.yaml` rempli
- [ ] `make new-saas` terminé sans erreur

- [ ] Les 3 URLs répondent avec le bon sha sur `/api/health`
- [ ] Premier backup prod visible dans `tonlab-backups-prod`
- [ ] Sondes ajoutées sur les 3 URLs
- [ ] SPF, DKIM et DMARC validés chez le fournisseur d'email

Chaque mois

- [ ] Mise à jour de Dokploy (backup avant, Setup Server de B si demandé)
- [ ] Test de restauration de la prod
- [ ] Revue du disque, des alertes et du budget AWS
- [ ] `curl http://IP_A:3000` échoue toujours

Sources

- Dokploy — Multi Server
- Dokploy — Multi-server deployments
- Dokploy — Production hardening
- Dokploy — Multi-tenancy (projets, environnements)
- Dokploy — Variables d'environnement
- Dokploy — API Compose
- Dokploy — Certificats
- Dokploy — Cloudflare
- Dokploy — Dépannage des domaines
- Dokploy — PostgreSQL et image personnalisée
- Dokploy — Backups de bases
- Dokploy — Backups Web Server
- Dokploy — Volume backups
- Dokploy — Nettoyage automatique (DeepWiki)
- Dokploy — Issue 2661, port 3000 et UFW
- Dokploy — Issue 947, retirer la publication du port 3000
- LumaDock — Commande de mise à jour de Dokploy
- GitHub — Environnements de déploiement
- HashiCorp — Backend S3 de Terraform
- Pulumi — Provider Dokploy